Combined Security Review

Internal + External + Wave 14 Close-out

Rumi Protocol v2

Window: 2026-04-16 → 2026-05-02

AVAI scope: e749620d

Rumi Protocol — Combined Security Review (Internal + External + Wave 14 close-out)

Status at close: every finding from the internal three-pass review and the AVAI external pre-audit is in one of three states: resolved on mainnet with a regression test, deferred-by-design until SNS migration, or accepted housekeeping. Eight HIGH-severity findings, all eight fixed. Thirty-six MEDIUM findings, all addressed. Twenty-five LOW findings, addressed or formally accepted. Eight additional findings from the AVAI external pre-audit closed in Wave 14a/b/c. **Current mainnet anchor:** `rumi_protocol_backend` `0xc6b999340c510ae32b77731ee54c6cabc112a14d201f7ea972ed1be30c8f1662` · `rumi_3pool` `0xfcf49d3042e376d6f0c7f04bdbfcac4c93fd242cbeda7e85c4800ba9f6591d97` · `rumi_stability_pool` `0xd8edd5aa...` (PR-a1's mirror update is dormant; ships on the next SP touch). **Review window:** 2026-04-16 (AVAI scope SHA `e749620d`) through 2026-05-02 (Wave 14c deploy).

Executive summary

Rumi Protocol underwent two parallel security reviews of its v2 stack and then a parity-and-close-out cycle to land both on the same outcome.

Internal three-pass review (anchor `28e9896`, 2026-04-22): 73 findings across 11 specialist analysis passes covering ~63k lines of Rust across 9 canisters and ~30k lines of TypeScript/Svelte across two frontends. Shipped in 13 numbered remediation waves over nine days, then a third pass that ran a drift check, hunted for orphans introduced during remediation, and closed three small loose ends (PR #146 merge, `SECURITY.md`, the analytics reinstall guard). Net result by 2026-05-02 morning: every finding closed.

AVAI external pre-audit (anchor `e749620d`, 2026-04-24): an automated pre-audit sweep against the Breitner IC Canister Security Guidelines and a CDP protocol domain checklist. It produced 20 numbered findings (4 IC-hygiene, 16 CDP-domain) plus narrative observations on access control, upgrade safety, and test coverage. Twelve overlapped with internal-review findings the team had already shipped fixes for; eight were genuinely net-new. **One AVAI finding (CDP-08) was reproduced live on IC mainnet from the anonymous principal**; that was already closed by the internal Wave-2 fix when AVAI's report landed.

Wave 14 close-out (2026-05-02): the eight net-new AVAI findings, packaged as four PRs (#157, #158, #160, #161, #162). Each PR was anchored to a fresh feature branch, gated by TDD-style audit-fence tests (15 new tests covering CDP-10/14/01, 4 covering CDP-03, 2 covering CDP-12, 2 covering CDP-09, plus B-01's 5 + CDP-16 doc-only), merged with a regular-merge commit (no squash), deployed via `dfx deploy --network ic --argument 'variant Upgrade ... description ...'` after the project's pre-deploy hook ran the full unit + PocketIC suite, and bake-watched.

The protocol has been continuously running on mainnet throughout the entire review and remediation cycle. No fund losses, no protocol pauses, no emergency interventions.

How the two reviews fit together

The internal review and the AVAI external pre-audit landed within two days of each other and were anchored to slightly different commits (`28e9896` vs. `e749620d`). They overlapped substantially because both were looking at the same threat surface: the IC's async-message model where state can change at every `await` point, the CDP-specific hazards around redemption/liquidation cascades, and the upgrade-safety / oracle-resilience layers that sit underneath those.

The right way to read them together is as an *N-of-2* check on the same codebase. Where the two reviews independently found the same bug, that's strong signal. Where they diverged, it's mostly because one had access to context the other didn't:

- The internal review knew the redemption tier system was a deliberate priority-1/2/3 design (RED-tier assignments documented in the team's working notes), so it didn't flag the apparent "redundant" tier-checking in `redeem_collateral`.

- The AVAI sweep didn't know the internal sorted-troves index was already in place behind LIQ-002 by the time AVAI scanned, so a couple of AVAI's "missing data structure" notes were already false-positives by AVAI's own scan date.
- Conversely, AVAI surfaced two findings the internal review hadn't framed quite as cleanly: CDP-14's `num_sources_used` floor on XRC metadata (the internal review caught the staleness gate but not the source-thinness gate), and CDP-12's "single timer chain" framing (the internal review had identified the trap-skips-downstream hazard but not packaged it as a structural fix).

After Wave 14, the union of both reviews is fully closed. There is no outstanding finding from either source that has not been resolved, deferred-by-design with explicit governance commitments, or accepted housekeeping with a documented watch threshold.

Internal three-pass review (recap)

The internal pass was structured deliberately to mimic a third-party engagement:

- **Anchored to a single commit** (`28e9896`) so every finding referenced a fixed state of the code, with file and line precision.
- **Eleven specialist analysis passes**, each scoped to one threat model (async-state races, oracle integrity, ICRC transfer hygiene, stable-memory upgrade safety, stability-pool accounting, redemption peg-defense, caller authorization, debt-and-interest accounting, liquidation mechanics, inter-canister-call failure modes, cycle/DoS exposure).
- **Findings recorded as structured JSON** with severity, mechanism, exploit scenario, and a recommended remediation. Severities calibrated to OWASP-style impact × likelihood.
- **Two-stage discovery**: a "dry run" of likely findings before the deep dive, followed by the primary review pass. Anything that survived both was real.
- **Three independent verification passes** after the fixes shipped, looking for incomplete fixes, regressions, and silent drift.

Findings by severity

- **HIGH (8):** issues with direct fund-loss potential or systemic protocol consequences.
- **MEDIUM (36):** correctness, accounting, or DoS issues that don't directly drain funds but degrade safety, fairness, or operational posture.
- **LOW (25):** hygiene, defense-in-depth, or operational items.
- **INFORMATIONAL (4):** observations, not issues.

High-severity findings — categories

Category	Findings	What was at stake
Caller authorization	2	Inter-canister boundary trust between protocol and stability pool; presence of developer-only `dev_*` endpoints in mainnet wasm
Inter-canister call failure	2	Stranded funds when a multi-step liquidation refund path's second call failed; pending-transfer retries that didn't distinguish ledger-fee changes from transient errors
ICRC transfer hygiene	2	Liquidation collateral and treasury withdrawal flows lacked `created_at_time` deduplication, opening a double-spend window on transient ledger reject
Liquidation mechanics	1	Concurrent partial-liquidations on the same vault could overwrite the first liquidator's payout entry
Stability pool accounting	1	A latent double-deduction in the pool's liquidation absorption path (proportional pre-deduct followed by another proportional deduct), draining depositor balances over many liquidations

Remediation waves (1 through 13)

Thirteen waves shipped between 2026-04-23 and 2026-05-01. Each wave is its own merged PR with deploy timestamp, pre/post module hashes, and a 24h bake-watch checklist:

Wave	Theme	Notable findings
1	Stability pool accounting	SP-001 double-deduction, SP-005 pool-balance reconciliation
2	Auth quick wins	AUTH-001 (SP boundary), AUTH-002 (`dev_*` removal), DOS-009
3	ICRC transfer hygiene	ICRC-001 to ICRC-005 backend half, ICC-006
4	Pending-transfer lifecycle	LIQ-001 keying, ICC-001/-002/-005/-007 refund queues
5	Oracle staleness + sanity	RED-001, LIQ-006, LIQ-007
6	Upgrade safety	UPG-001/-002/-004/-006
7	Interest accounting	INT-001/-003/-004
8 (a-e)	Liquidation invariants	LIQ-002 sorted-troves, LIQ-003 min-debt, LIQ-004 burn-proof, LIQ-005 deficit account
10	Mass-liquidation breaker	LIQ-008
11	Bot auto-cancel collateral verification	BOT-001
12	Bot explicit-cancel gate	BOT-001b (ICC-008)
13	Bot swap-failure error propagation	BOT-002

Plus three follow-up waves between 2026-05-01 and 2026-05-02 morning:

Wave	Findings	What shipped
INT-002 + INT-006	INT-002 (MED), INT-006 (LOW)	Snapshot-then-decrement with saturating restore on `Err` for both interest harvest and treasury drain
RED-002 + RED-003	RED-002 (MED), RED-003 (MED)	`validate_mode()` at the top of both redemption endpoints; redemption shortfall routes through Wave-8e deficit account
9a	DOS-001/-003/-004/-008	Pagination on every previously-unbounded query
9b	DOS-006/-007	Aggregate-query 5-second TTL cache for `get_protocol_status` and `get_treasury_stats`
9c	DOS-005	`check_vaults` sharded to at-risk CR band via the Wave-8a sorted-troves index
9d	DOS-010, DOS-011	Analytics pull-cycle stagger; XRC timer lifecycle gate on `add/disable_collateral_token`
ICRC-005 frontend	ICRC-005 (frontend)	Live `icrc1_fee()` query via `ledgerFeeService.ts` with 5-min cache

Third pass — drift check + orphan hunt

The third pass independently re-verified the second-pass "Resolved-Confirmed" rows against current code. AUTH-001, AUTH-002, LIQ-001, LIQ-002, LIQ-005, UPG-006 all confirmed clean. The pass also looked for new issues introduced *during* the remediation cycle (every new admin endpoint developer-gated; every new state field carries `#[serde(default)]` ; no backwards-incompatible candid changes; no stale TODOs). All eight tracked canister hashes match the documented post-deploy hashes from the wave memory files. No undocumented redeploys, no rollbacks, no drift.

Three small loose ends were closed before declaring the cycle complete: PR #146 (the prepared-but-unmerged ICC-002 + BOT-002 audit POCs) merged at `4f93de5`; `SECURITY.md` added at the repo root; `rumi_analytics::init` reinstall guard added in PR #155 / `480c5c7`.

Final tally

Status	Count	Notes
Resolved on mainnet with regression-test fence	60	All 8 HIGH, 30 of 36 MEDIUM, 22 of 25 LOW
Deferred-by-design until SNS migration	5	AUTH-003 single-controller posture; AUTH-004/-007/-008/-009 admin-rotation hygiene
Accepted housekeeping	4	Operational items where current scale doesn't justify the change; explicit watch thresholds
Informational	4	Observations recorded but no action required
Total	**73**	

Plus three audit-discovered follow-on findings (BOT-001, BOT-001b/ICC-008, BOT-002) all resolved during Waves 11-13 with their own test fences.

AVAI external pre-audit

What AVAI is

AVAI is an automated pre-audit sweep that uses LLM-assisted routing, source-code analysis, and the Breitner IC Canister Security Guidelines plus a CDP protocol domain checklist to surface known-shape bugs (await-point interleaving, `created_at_time` drift, oracle staleness, redemption/liquidation path inconsistencies, cycle DoS).

The honest framing in AVAI's own report: **not a substitute for a full human audit by an IC-specialised CDP auditor**. Findings should be triaged by the team and then reviewed by a human auditor before mainnet release. **One AVAI finding (CDP-08) was reproduced live on IC mainnet from the anonymous principal.** All other findings were source-level analysis at the audited commit (`e749620d`); PoC sketches were given for every MEDIUM-or-higher and runnable by the team in PocketIC.

Scope: ~24k LoC of Rust across 6 canisters: `rumi_protocol_backend`, `stability_pool`, `liquidation_bot`, `rumi_3pool`, `rumi_amm`, `rumi_treasury`. Out of scope: cryptographic primitives, candid encoding, frontend code, economic parameter tuning, anything requiring live mainnet reproduction.

AVAI findings catalogue

Part I — IC Canister Hygiene (4 findings)

ID	Title	Severity	Disposition
B-01	No `PoolGuard` : concurrent messages at await points corrupt stableswap invariant	MED	**Closed in Wave 14a PR-a1** (#157)
B-02	`created_at_time: None` in all ICRC-1/-2 transfer calls	HIGH	Already closed by internal Wave 3 (ICRC-001/-002/-003/-004)
B-03	`post_upgrade` validation is log-only — orphaned vaults do not abort upgrade	MED	Already closed by internal Wave 6 (UPG-001/-002)
B-04	`dev_force_bot_liquidate` unconditionally compiled, no cap, no cooldown, no on-chain event	HIGH	Already closed by internal Wave 2 (AUTH-002)

Part II — CDP Protocol Domain Layer (16 findings)

ID	Title	Severity	Disposition
CDP-01	XRC oracle silent on failure — no ReadOnly fallback within 10-minute window	MED	**Closed in Wave 14a PR-a2** (#160)
CDP-02	Redemption Margin Ratio divergent paths: latent structural risk	LOW	Internal review confirmed deliberate by-design; the two paths apply RMR at different stages of the same calculation, not redundantly
CDP-03	Per-collateral base-rate writes hit global state	LOW	**Closed in Wave 14b** (#161)
CDP-04	Interest opt-out timing race with liquidation notification	LOW	Already closed by internal INT-002 follow-up (snapshot-then-decrement with `Err` rollback)
CDP-05	Stability pool balance drifts negative on `Err(call_error)`	MED	Already closed by internal Wave 1 (SP-005 reconciliation)
CDP-06	ckStable stranded in bot on 3pool failure, no rescue path	MED	Already closed by internal Wave 13 (BOT-002 cancel/return error propagation)
CDP-07	Timer cycle cost scales O(n) with collateral count	INFO	Accepted; current collateral count (≤8) keeps cycle cost in budget; revisit at 16+ collaterals
CDP-08	`notify_liquidatable_vaults` caller check is log-only — **LIVE-EXPLOITABLE ON MAINNET**	HIGH	Already closed by internal Wave 2 (AUTH-001) — the caller gate was tightened from log-only to a hard reject before AVAI's report landed
CDP-09	`global_close_requests` O(N) scan on every `close_vault`	INFO	**Closed in Wave 14c** (#162)
CDP-10	Liquidation cascade dead-end: `sp_attempted_vaults` set before call resolves	MED	**Closed in Wave 14a PR-a2** (#160)
CDP-11	Inconsistent rounding-dust policy in stability pool	LOW	Internal review confirmed deliberate; the SP rounds dust toward depositors (worst case = 1 e-8 lost); per-call dust never compounds
CDP-12	Timer tick bundles oracle + accrual + cascade: partial-failure risk	LOW	**Closed in Wave 14b** (#161)
CDP-13	No `--mode reinstall` guard: silent stable-memory wipe possible	MED	Already closed by internal Wave 6 (UPG-006) and the Wave-14-era follow-up adding the guard to `rumi_analytics`
CDP-14	XRC oracle: single source, no `num_sources_used` check, manipulation cost unbounded at low-liquidity windows	MED	**Closed in Wave 14a PR-a2** (#160)
CDP-15	`bot_claim_liquidation` : concurrent-call race across two await points	MED	Already closed by internal Wave 11 (BOT-001 collateral-return verification)
CDP-16	`validate_call().await` window allows state drift before liquidatability re-check	INFO	**Closed in Wave 14c** (#162) — doc comment on `validate_call` calling out the suspension boundary

Parts III through V (access control, upgrade safety, test coverage) are observation-and-recommendation prose rather than numbered findings. Their material recommendations were already closed by the internal review's Waves 2, 6, and the audit-fence test discipline that runs on every PR.

NEW-01 (oracle aggregation): explicitly out of Wave 14 scope

AVAI flagged a structural NEW-01 on multi-source oracle aggregation for thin-liquidity collateral. Wave 14 explicitly does NOT close this. The current operative control is per-collateral debt ceilings — BOB at $500 and EXE at $500 — which bound the manipulation risk. A second oracle source becomes the right answer when the protocol takes on a non-promotional, organic, thin-liquidity collateral with a meaningful debt ceiling. Tracked as the ORACLE-001 backlog item; ride along with the SNS migration or the next thin-liquidity collateral addition.

Net AVAI footprint

- 8 net-new findings closed in Wave 14a / 14b / 14c.
- 9 findings already closed by the internal three-pass review.
- 3 findings confirmed deliberate-by-design (CDP-02, CDP-07, CDP-11).
- 1 finding deferred-by-design (NEW-01 / ORACLE-001).

Wave 14 close-out

Sub-wave 14a (the four MEDIUM-tier findings) was packaged as two PRs to keep blast radius small. Sub-wave 14b (two LOW) and 14c (two INFO) shipped after 14a baked.

Wave 14a PR-a1 — B-01 (`rumi_3pool` reentrancy guard)

PR #157 plus PR #158 (defensive SP mirror update so its decoder for `ThreePoolError` accepts the new `PoolLocked` variant).

The fix: ports the `PoolGuard` pattern from `rumi_amm` to `rumi_3pool`. Single canister-wide flag (one pool per canister, vs. `rumi_amm`'s per-pool `BTreeSet`), released via `Drop` so it survives traps. Acquired before the first `await` in six entry points — `swap`, `add_liquidity`, `remove_liquidity`, `remove_one_coin`, `donate`, `authorized_redeem_and_burn`. Returns the new `ThreePoolError::PoolLocked` so callers can retry rather than block.

Before the fix: two concurrent swap callers could both read the same pre-balances, both compute the same output, and both transfer that output, leaking LP value. AVAI's trace estimated ~0.05% per concurrent pair at 10k-token depth.

Test fence: `src/rumi_3pool/tests/audit_pocs_b_01_pool_guard.rs` (5 tests) — concurrent-swap serialization (the core regression test, fails before the fix with both swaps returning `Ok(9998999)`, passes after with one returning `PoolLocked`), release-on-Ok, release-on-Err, and add_liquidity/swap concurrent coverage. Plus a unit test for the guard's exclusivity in `pool_guard.rs`.

Mainnet (`rumi_3pool` = `fohh4-yyaaa-aaaap-qtkpa-cai`): pre-deploy `0xf7e39f8d…`, post-deploy `0xfcf49d30…`. Sanity-checked against live swap traffic post-deploy: 6 swaps + 2 add_liquidity + 1 remove_one_coin all succeeded, zero `PoolLocked` errors, virtual_price drifted ~+1e-9 (the fee accrual that the leak was previously bleeding off).

Wave 14a PR-a2 — CDP-10, CDP-14, CDP-01 (backend bundle)

PR #160. Three independent backend hardening fixes bundled together because they all touch the same canister and address related mid-tick failure modes.

CDP-10: `record_sp_notification_result` (pure-state helper in `lib.rs`). On `Ok` it inserts the dispatched vault ids into `sp_attempted_vaults`; on `Err` it leaves the set unchanged and emits `Event::StabilityPoolCallFailed` so external liquidators polling `get_events` can react. Pre-fix the synchronous insert before the spawn meant a transport `Err` permanently blacklisted vaults from the SP path even though no liquidator was ever notified.

CDP-14: new configurable `min_xrc_sources_used: u32` State field, default 3 (defined as `xrc::MIN_XRC_SOURCES`). Pure helper `xrc_metadata_meets_source_floor`. Wired into both `xrc::fetch_icp_rate` and `management::fetch_collateral_price`. Reads `metadata.base_asset_num_received_rates` (the actual `ic-xrc-types` field; AVAI's "num_sources_used" was approximate). Rejection emits `Event::OracleSourceCountInsufficient` and the cached price stays in place. Developer-gated setter `set_min_xrc_sources_used(value: u32)` is on the candid surface.

CDP-01: new State fields `consecutive_xrc_failures: u64` and `mode_triggered_by_oracle: bool`, both `#[serde(default)]` for clean upgrade decode of pre-Wave-14 snapshots. Pure helpers `note_xrc_failure` / `note_xrc_success`. After `MAX_CONSECUTIVE_XRC_FAILURES = 3` consecutive failures, flips to `Mode::ReadOnly` and emits `Event::OracleCircuitBreaker`. Successful fetch resets the counter and clears ReadOnly only when oracle-triggered (operator-set ReadOnly persists across oracle recovery).

Test fence: 4 tests for CDP-10, 4 for CDP-14, 7 for CDP-01 — 15 new audit fences.

Mainnet (`rumi_protocol_backend` = `tfesu-vyaaa-aaaap-qrd7a-cai`): pre-deploy `0x5ef41221…`, post-deploy `0xf226c873…`.

Wave 14b — CDP-03 + CDP-12

PR #161. Two independent fixes shipped together.

CDP-03: `redeem_collateral` and the spillover arm of `redeem_reserves` now go through `s.get_redemption_fee_for(&ct, ...)` and write the post-redemption base rate via the new `record_per_collateral_redemption_fee` helper. The legacy global `s.current_base_rate` and `s.last_redemption_time` are no longer mutated by the redemption path (kept for aggregate-display callers). A redemption against ckBTC no longer corrupts the rate that prices ICP redemptions.

CDP-12: the pre-Wave-14b 300s XRC tick chained `fetch_icp_rate → interest accrual → harvest → treasury drains → flush → check_vaults → bot/SP spawn → refresh snapshots`. A trap anywhere in the chain skipped everything downstream silently for the next 5 minutes. Now three independent `set_timer_interval` callbacks:

- **Timer A** (300s, unchanged from legacy): `fetch_icp_rate` (XRC fetch + price + CDP-14 + CDP-01).
- **Timer B** (60s, new): `interest_and_treasury_tick` (interest accrual + harvest + treasury drains + flush). Cheap; runs more often without cycle pain.
- **Timer C** (300s, new): `vault_check_tick` (check_vaults + aggregate-snapshot refresh). Matches the legacy cadence so liquidation latency is unchanged.

IC's per-message trap isolation gives the "fail-isolated" runtime behavior the plan asked for; no application-level `catch_unwind` is needed.

Cadence deviation from plan: the AVAI plan suggested Timer A at 60s. We kept it at 300s to avoid a 5x XRC cycle increase that warrants its own ops review. The split itself satisfies CDP-12's isolation goal; cadence tuning can ride a future PR via setters.

Test fence: 4 tests for CDP-03, 2 for CDP-12 — 6 new audit fences. CDP-12's tests verify the three entry points exist with `pub async fn () -> ()` signatures and the intervals have sensible defaults.

Mainnet: pre-deploy `0xf226c873…`, post-deploy `0x619c6deb…`.

Wave 14c — CDP-09 + CDP-16 (housekeeping)

PR #162. Two INFO-tier polish items.

CDP-09: `global_close_requests` is now a `VecDeque<u64>`. Cleanup walks `partition_point` then `drain(..idx)` (O(log N + K)) instead of `retain` (O(N) over up to 30k entries every close call). Per-user `close_vault_requests` stays `Vec<u64>` since per-user lists are tiny (5/min, 60/day).

CDP-16: doc comment on `validate_call` calling out the `ensure_fresh_price().await` suspension point — callers must re-read state after `validate_call().await` returns. Five-minute doc edit; no behavior change.

Test fence: 2 compile-time tests for CDP-09 (the field's exact type plus the empty-deque default). CDP-16 is doc-only.

Mainnet: pre-deploy `0x619c6deb…`, post-deploy `0xc6b99934…`.

Wave 14 deploy ritual

Each PR followed the same ritual:

1. Feature branch from `main`.
2. TDD-style audit-fence test written first, watched fail (RED), then implementation, watched pass (GREEN).

3. Pre-deploy verification: `cargo test --lib` (366 tests workspace-wide), `cargo test --test pocket_ic_tests --test pocket_ic_3usd --test integration_test --test pocket_ic_analytics` (82 tests across 4 hook-gated suites), plus the new audit fences.
4. PR opened, mergeability confirmed, merged with `gh pr merge --merge` (regular merge commit, never squash).
5. Capture pre-deploy mainnet module hash via `dfx canister --network ic info`.
6. Deploy with `dfx deploy --network ic <canister> --argument 'variant Upgrade ... description ...'`. Pre-deploy hook runs the test suite again and saves the passing commit hash to `.claude/hooks/.last_test_pass`.
7. Capture post-deploy module hash. Sanity-check via a query that exercises the new code path (`get_protocol_status`, `get_pool_status`).
8. Memory note written documenting hashes, test count, and bake-watch checklist.
9. 24h bake (or shortened spot-check when the canister is healthy and traffic-active).

The pre-deploy hook (`.claude/hooks/pre-deploy-test.sh`) gates every backend deploy. Frontend-only deploys skip it.

Combined findings tally

Source	Total findings	Resolved	Deferred-by-design	Accepted housekeeping	Informational / by-design
Internal three-pass	73	60	5	4	4
AVAI (net-new)	8	8	0	0	0
AVAI (overlap with internal)	9	9	0	0	0
AVAI (deferred-by-design)	1	0	1 (NEW-01 / ORACLE-001)	0	0
AVAI (confirmed deliberate)	3	0	0	0	3 (CDP-02, CDP-07, CDP-11)
Audit-discovered follow-on	3	3	0	0	0
Combined	**97**	**80**	**6**	**4**	**7**

Every "Resolved" row has a regression-test fence that runs on every CI cycle and on every pre-deploy hook. Every "Deferred" row has an explicit governance commitment in `SECURITY.md` (or in the project memory for the AVAI-deferred ORACLE-001 backlog). Every "Accepted" row has a documented watch threshold.

Mainnet hash inventory at close

Canister	Canister ID	Module hash at close	Last wave touched
`rumi_protocol_backend`	`tfesu-vyaaa-aaaap-qrd7a-cai`	`0xc6b999340c510ae32b77731ee54c6cabc112a14d201f7ea972ed1be30c8f1662`	Wave 14c
`rumi_3pool`	`fohh4-yyaaa-aaaap-qtkpa-cai`	`0xfcf49d3042e376d6f0c7f04bdbfcac4c93fd242cbeda7e85c4800ba9f6591d97`	Wave 14a PR-a1
`rumi_stability_pool`	`tmhzi-dqaaa-aaaap-qrd6q-cai`	`0xd8edd5aa…` (pre-Wave-14; PR-a1's mirror update is dormant, ships on next SP touch)	Wave 9a
`rumi_treasury`	`tlg74-oiaaa-aaaap-qrd6a-cai`	(pre-Wave-14)	—
`rumi_amm`	`ijlzs-2yaaa-aaaap-quaaq-cai`	(pre-Wave-14)	—
`liquidation_bot`	`nygob-3qaaa-aaaap-qttcq-cai`	`0x820d0d72…`	Wave 13
`rumi_analytics`	`dtlu2-uqaaa-aaaap-qugcq-cai`	`0x747ac92a…`	Wave 9d
`vault_frontend`	`tcfua-yaaaa-aaaap-qrd7q-cai`	`0x423f20ee…` (asset state changed; wasm hash unchanged)	ICRC-005 frontend

All hashes verifiable via `dfx canister --network ic info <name>`.

What's next

This is the cleanest shape the protocol stack has been in since v2 went live. Both the internal review and the AVAI external pre-audit are fully closed. Every regression has a test fence; every deferral has an explicit commitment.

The next routine review is scheduled for early Q3 2026, with two specific triggers that would advance it earlier:

1. **The SNS migration project starting.** SNS handoff is itself an admin-rotation event, and a fresh review at that point will fold in the four governance-hygiene findings currently parked under "deferred to SNS" — confirming on-chain that controller authority, two-phase admin rotation, and proposal-gated config changes all behave as designed. AVAI's NEW-01 oracle-aggregation deferral also rides along with this trigger.
2. **A new collateral type or significant TVL inflection.** New collaterals introduce XRC plumbing, oracle freshness assumptions, and liquidation-bot integration that should be revalidated against the threat model. Significant TVL inflection is the trigger for revisiting the deferred housekeeping items (event-log eviction, pre-upgrade serialization layout) where the cost-benefit tradeoff changes with scale.

In the meantime, every deploy continues to run pre-deploy unit and integration tests via the project's hook system; module hashes are recorded on-chain at deploy time and cross-referenced in commit messages; and the protocol's mainnet behavior is continuously instrumented through the analytics canister for the kind of drift that doesn't show up in static review.

How to verify this independently

Findings, fixes, deployed module hashes, and test fences are all traceable through the public commit history of `RumiLabsXYZ/rumi-protocol-v2`:

- Wave-by-wave PRs link to specific commits, candid changes, and test additions.
- Each merged PR has a deploy-time entry in the project memory at `~/.claude/projects/.../memory/` documenting pre/post hashes and bake-watch results (those notes are not in the public repo since they include operational specifics, but every fact in them is independently verifiable from the public commit + on-chain hash).
- The full unit + PocketIC test suite is reproducible with `cargo test --lib` (workspace) and `POCKET_IC_BIN=./pocket-ic cargo test --test pocket_ic_tests --test pocket_ic_3usd --test integration_test --test pocket_ic_analytics`. The 19 audit-fence binaries from Wave 14 (5 in 3pool, 14 in backend) run via `cargo test --test 'audit_pocs_*'`.
- Mainnet hashes verifiable with `dfx canister --network ic info <canister>`.
- Reach out via the responsible-disclosure contact in `SECURITY.md` with questions, follow-up findings, or independent verification work.

— Rumi Protocol team, 2026-05-02